

ЛР 1: [C++ UNIX]:
UNIX знакомство: useradd, nano, chmod, docker, GIT, CI, CD

Группа: Z3344
Студент: Глазко Лана
2025, 3 курс

1 Цель работы

Познакомиться с основами администрирования программных комплексов в ОС семейства UNIX, продемонстрировать особенности виртуализации и контейнеризации, продемонстрировать преимущества использования систем контроля версий (на примере GIT)

2 Задачи, решаемые при выполнении работы

- ОС: Работа в ОС, использование файловой системы, прав доступа, исполнение файлов
- КОНТЕЙНЕР: docker build / run / ps / images
- GIT: GitHub / GitLab, в котором будут содержаться все выполненные ЛР

3 Решение задачи 1. [ОС] Работа в ОС, использование файловой системы, прав доступа, исполнение файлов

1.1. Создание директорий folder_max и folder_min в папке /usr/local

Получаем права суперпользователя:

```
sudo -i
```

Создаём директории:

```
mkdir /usr/local/folder_max  
mkdir /usr/local/folder_min
```

Проверяем, что директории созданы:

```
ls -ld /usr/local/folder_*
```

Результат выполнения:

```
drwxr-xr-x 2 root root 4096 Apr 12 21:56 /usr/local/folder_max  
drwxr-xr-x 2 root root 4096 Apr 12 21:56 /usr/local/folder_min
```

1.2. Создание двух групп пользователей: group_max и group_min

Создаём группу group_max:

```
sudo groupadd group_max
```

Создаём группу `group_min`:

```
sudo groupadd group_min
```

Проверяем, что группы добавлены:

```
getent group | grep group_
```

Результат:

```
group_max:x:1001:  
group_min:x:1002:
```

1.3. Создание пользователей и добавление их в соответствующие группы

Создаём пользователя `user_max_1` и добавляем его в группу `group_max`:

```
sudo useradd -m -G group_max user_max_1
```

Создаём пользователя `user_min_1` и добавляем его в группу `group_min`:

```
sudo useradd -m -G group_min user_min_1
```

Устанавливаем пароли для новых пользователей:

```
sudo passwd user_max_1  
sudo passwd user_min_1
```

Проверка групп пользователя:

```
groups user_max_1  
groups user_min_1
```

Результат:

```
user_max_1 : user_max_1 group_max  
user_min_1 : user_min_1 group_min
```

1.4. Установка прав доступа для групп

`group_max` → полный доступ к `folder_max` и `folder_min`

`group_min` → полный доступ только к `folder_min`

Даём группе `group_max` полный доступ к `folder_max`:

```
sudo setfacl -m g:group_max:rwX /usr/local/folder_max
```

Даём группе `group_max` полный доступ к `folder_min`:

```
sudo setfacl -m g:group_max:rwX /usr/local/folder_min
```

Даём группе `group_min` полный доступ к `folder_min`:

```
sudo setfacl -m g:group_min:rwX /usr/local/folder_min
```

Проверка прав доступа для папок `folder_max` и `folder_min`:

```
getfacl /usr/local/folder_max  
getfacl /usr/local/folder_min
```

Результат (пример для `folder_min`):

```
# file: usr/local/folder_min
# owner: root
# group: root
user::rwx
group::r-x
group:group_max:rwx
group:group_min:rwx
mask::rwx
other::r-x
```

Результат (пример для folder_max):

```
# file: usr/local/folder_max
# owner: root
# group: root
user::rwx
group::r-x
group:group_max:rwx
mask::rwx
other::r-x
```

1.5. Создание и исполнение скрипта, записывающего дату/время в текущую директорию (пользователем из группы group_max)

Войти под пользователем user_max_1:

```
su - user_max_1
```

Перейти в директорию:

```
cd /usr/local/folder_max
```

Создать скрипт:

```
nano write_date_local.sh
```

Вставить в редактор nano следующий код:

```
#!/bin/bash
date >> output.log
```

Сделать скрипт исполняемым:

```
chmod +x write_date_local.sh
```

Запустить скрипт:

```
./write_date_local.sh
cat output.log
```

Результат:

```
Thu Apr 17 19:41:54 MSK 2025
```

1.6. Создание и исполнение скрипта из folder_max, записывающего дату/время в folder_min

Переход в директорию:

```
cd /usr/local/folder_max
```

Создание скрипта:

```
nano write_to_min.sh
```

Вставить в файл следующий код:

```
#!/bin/bash  
date >> /usr/local/folder_min/output.log
```

Сделать скрипт исполняемым:

```
chmod +x write_to_min.sh
```

Запуск скрипта:

```
./write_to_min.sh
```

Проверка результата:

```
cat /usr/local/folder_min/output.log
```

Результат:

```
Thu Apr 17 20:01:11 MSK 2025
```

1.7. Исполнение скрипта пользователем _min

Для начала нужно предоставить группе group_min права на запись в папку folder_min и файл output.log.

Даем группе group_min права на запись в папку folder_min:

```
sudo setfacl -m g:group_min:rwX /usr/local/folder_min
```

Даем группе group_min права на запись в файл output.log:

```
sudo setfacl -m g:group_min:rw /usr/local/folder_min/output.log
```

Исполняем скрипт от имени пользователя user_min_1:

```
su - user_min_1 -c "bash /usr/local/folder_max/write_to_min.sh"
```

Результат:

```
Thu Apr 17 20:31:54 MSK 2025
```

1.8. Создание и исполнение скрипта из folder_min, записывающего дату/время в folder_max

Пользователь user_min_1 создаёт скрипт write_to_max.sh в директории /usr/local/folder_min, который записывает текущую дату и время в файл output.log внутри /usr/local/folder_max.

Создание скрипта пользователем user_min_1

```
cd /usr/local/folder_min
nano write_to_max.sh
```

Содержимое скрипта:

```
#!/bin/bash
date >> /usr/local/folder_max/output.log
```

Сделать скрипт исполняемым:

```
chmod +x write_to_max.sh
```

Далее меняем пользователя и пользователь user_max_1 выполняет данный скрипт:

```
bash /usr/local/folder_min/write_to_max.sh
```

Результат проверяем:

```
cat /usr/local/folder_max/output.log
```

Результат:

```
Thu Apr 24 19:41:54 MSK 2025
Tue May 6 18:12:50 MSK 2025
```

1.9 Сохранение прав доступа (ACL) и информации о содержимом папок folder_min и folder_max

Были выполнены команды для сохранения ACL-прав и информации о содержимом каталогов /usr/local/folder_min и /usr/local/folder_max:

```
getfacl -R /usr/local/folder_min > folder_min_acl.txt
getfacl -R /usr/local/folder_max > folder_max_acl.txt
ls -lR /usr/local/folder_min > folder_min_ls.txt
ls -lR /usr/local/folder_max > folder_max_ls.txt
```

Содержимое файлов было проверено с помощью команды:

```
cat folder_min_acl.txt
cat folder_max_acl.txt
cat folder_min_ls.txt
cat folder_max_ls.txt
```

Пример содержимого файла folder_min_acl.txt:

```
# file: usr/local/folder_min
# owner: root
# group: root
user::rwx
group::r-x
group:group_max:rwx
group:group_min:rwx
```

```
mask::rwx
other::r-x

# file: usr/local/folder_min/output.log
# owner: user_max_1
# group: user_max_1
user::rw-
group::rw-
group:group_min:rw-
mask::rw-
other::r--
```

Пример содержимого файла `folder_max_acl.txt`:

```
# file: usr/local/folder_max
# owner: root
# group: root
user::rwx
group::r-x
group:group_max:rwx
mask::rwx
other::r-x

# file: usr/local/folder_max/output.log
# owner: user_max_1
# group: user_max_1
user::rw-
group::rw-
other::r--

# file: usr/local/folder_max/write_date_local.sh
# owner: user_max_1
# group: user_max_1
user::rwx
group::rwx
other::r-x

# file: usr/local/folder_max/log_time_min.sh
# owner: user_max_1
# group: user_max_1
user::rwx
group::rwx
other::r-x

# file: usr/local/folder_max/write_to_min.sh
# owner: user_max_1
# group: user_max_1
user::rwx
group::rwx
other::r-x

# file: usr/local/folder_max/log_time.sh
# owner: user_max_1
# group: user_max_1
```

```
user::rwx
group::rwx
other::r-x
```

Пример содержимого folder_min_ls.txt:

```
/usr/local/folder_min:
total 8
-rw-rw-r--+ 1 user_max_1 user_max_1 58 Apr 24 20:31 output.log
-rwxrwxr-x  1 user_min_1 user_min_1 53 May  6 18:00 write_to_max.sh
```

Пример содержимого folder_max_ls.txt:

```
/usr/local/folder_max:
total 20
-rwxrwxr-x 1 user_max_1 user_max_1 31 Apr 12 21:28 log_time.sh
-rwxrwxr-x 1 user_max_1 user_max_1 73 Apr 12 21:56 log_time_min.sh
-rw-rw-r-- 1 user_max_1 user_max_1 58 May  6 18:12 output.log
-rwxrwxr-x 1 user_max_1 user_max_1 31 Apr 24 19:41 write_date_local.sh
-rwxrwxr-x 1 user_max_1 user_max_1 53 Apr 24 20:00 write_to_min.sh
```

Таким образом, были корректно сохранены и проверены как стандартные права доступа (права владельца, группы, других пользователей), так и расширенные права (ACL) для нужных директорий и файлов.

4 Решение задачи 2. [КОНТЕЙНЕР] docker build / run / ps / images

2.1 Создание скрипта для записи текущей даты и времени в файл output.log

Для выполнения задания создан скрипт `write_time.sh`, который записывает текущие дату и время в файл `output.log` в текущей директории. Ниже приведены все шаги создания и проверки работы скрипта:

Перейти в домашнюю директорию:

```
cd ~
```

Создаем файл скрипта с помощью редактора `nano`:

```
nano datetime\_script.sh
```

Создан скрипт `datetime_script.sh`:

```
#!/bin/bash
date >> output.log
echo "Current_date/time_written_to_output.log" >> output.log
```

Проверка работы скрипта:

```
./datetime_script.sh
cat output.log
```

Результат:

```
Fri May  9 19:35:45 MSK 2025
Current date/time written to output.log
```

2.2. Сборка Docker образа

Создан Dockerfile:

```
FROM ubuntu:latest
# обновить индекс пакетов (apt-get update) и установить nano
RUN apt-get update && apt-get install -y nano
COPY datetime_script.sh /scripts/datetime_script.sh
# скопировать файл datetime_script.sh из текущей директории на хосте в директорию /scripts
  внутри образа
# дать скрипту права на исполнение внутри образа.
RUN chmod +x /scripts/datetime_script.sh
# установить рабочую директорию внутри контейнера: все команды будут выполняться из /
  scripts
WORKDIR /scripts
```

При сборке возникла и решена проблема прав доступа:

```
# добавить текущего пользователя ($USER) в группу docker, чтобы иметь права на запуск
  Docker без sudo
sudo usermod -aG docker $USER
newgrp docker
# построить Docker-образ на основе текущей директории (.), используя Dockerfile;
  -t datetime_image присваивает образу имя datetime_image
docker build -t datetime_image .
```

2.3-2.4. Запуск контейнера и выполнение скрипта

```
docker run -it --name datetime_container datetime_image /bin/bash
cd /scripts
./datetime_script.sh
cat output.log
```

Результат:

```
Fri May  9 16:39:42 UTC 2025
Current date/time written to output.log
```

2.5. Список пользователей в образе

```
docker run datetime_image bash -c "getent _passwd"
```

Вывод (сокращённый):

```
# имя_пользователя:хэш_пароля:UID:GID:комментарий:домашняя_директория:оболочка
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
ubuntu:x:1000:1000:Ubuntu:/home/ubuntu:/bin/bash
```


5 Решение задачи 3. [Система контроля версий GIT]

3.1-3.2. Инициализация репозитория и создание структуры проектов

Для организации рабочего пространства был создан Git-репозиторий со следующей структурой:

Инициализация нового репозитория Стандартная структура для каждого проекта Базовые файлы документации

```
git init
mkdir -p lab_{01,02}/{build,src,doc,cmake}
touch lab_{01,02}/doc/README.md
```

Пояснение:

- Директория `build` будет содержать файлы сборки
- `src` - исходный код программ
- `doc` - документацию (включая отчеты по ЛР)
- `cmake` - файлы для системы сборки CMake

3.3. Настройка workflow с ветками разработки

Была реализована модель ветвления, соответствующая промышленным стандартам:

```
# Основная ветка (стабильная версия)
git branch -M main

# Ветка разработки (основная рабочая ветка)
git checkout -b dev

# Ветка тестирования (staging)
git checkout -b stg

# Продуктовая ветка (production)
git checkout -b prd

# Возврат к разработке
git checkout dev
```

Схема workflow:

Ветка	Назначение
main	Стабильная версия (только релизные коммиты)
dev	Основная разработка (интеграция функций)
stg	Тестирование и проверка (staging environment)
prd	Продуктовая версия (production environment)

3.4-3.5. Автоматизация процессов переноса кода

Для минимизации ручных операций созданы скрипты:

`dev_to_stg.sh` - перенос кода из разработки в тестирование:

```
#!/bin/bash
# Безопасный переход на ветку тестирования
git checkout stg || exit 1

# Слияние с веткой разработки
git merge dev -m "[CI] Автоматический перенос dev->stg $(date +%F)"

# Создание versionного тега
git tag -a "stg-$(date +%Y%m%d)" -m "Сборка для тестирования"

# Публикация изменений
git push origin stg --tags

# Возврат в исходную ветку
git checkout dev
```

stg_to_prd.sh - выкатывание версии в продакшен:

```
#!/bin/bash
# Проверка наличия изменений
if [ -z "$(git diff stg prd)" ]; then
    echo "Нет изменений для переноса"
    exit 0
fi

git checkout prd
git merge --no-ff stg -m "[RELEASE] $(date +%Y-%m-%d)"
git tag -a "prd-$(date +%Y%m%d)" -m "Продуктовый релиз"
git push origin prd --tags
git checkout dev
```

5.1 Интеграция с GitHub

Настроена безопасная аутентификация через SSH:

```
# Генерация криптографически безопасного ключа
ssh-keygen -t ed25519 -C "lana@smdh.ru" -f ~/.ssh/github_key -N ""

# Добавление ключа в ssh-agent
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/github_key

# Настройка удаленного репозитория
git remote add origin git@github.com:sudbreak/c-.git
git push -u origin --all # Отправка всех веток
git push origin --tags   # Отправка всех тегов
```

6 Решение задачи 4. [Финализация и сохранение работы]

Финальный коммит

Перед завершением работы все изменения были зафиксированы:

```
# Индексация всех измененных файлов
git add .

# Создание коммита с полным описанием
git commit -m "Завершение_ЛР_#1._Реализованы:
#_Настройка_прав_доступа_в_UNIX
#_Работа_с_Docker-контейнерами
#_Настройка_Git_workflow
#_Автоматизация_деплойа"

# Публикация изменений
git push origin dev
```

Процесс деплоя через CI/CD

Выполнено развертывание через pipeline:

```
# Тестирование сборки
./dev_to_stg.sh
# Результат:
# [stg-20250509] Успешная сборка для тестирования

# Продуктовый релиз
./stg_to_prd.sh
# Результат:
# [prd-20250509] Продуктовая версия опубликована
```

Верификация результатов

Итоговое состояние репозитория:

```
* 6f0ef78 (dev, stg, prd) Добавлены скрипты деплоя
* 292227b Настройка CI/CD pipeline
* 6274115 Инициализация проекта
```

Список версионных тегов:

```
prd-20250509    Продуктовый релиз v1.0
stg-20250509    Тестовая сборка 2025-05-09
```

7 Заключение

В ходе выполнения лабораторной работы были достигнуты следующие результаты:

1. Администрирование UNIX:

- Настроена система пользователей и групп
- Реализовано разграничение прав через ACL
- Созданы и протестированы исполняемые скрипты

2. Работа с Docker:

- Собран кастомный образ на базе Ubuntu
- Настроена передача скриптов в контейнер
- Проверена работа приложения в изолированной среде

3. Система контроля версий:

- Реализована промышленная модель ветвления
- Настроена автоматизация процессов CI/CD
- Обеспечена интеграция с GitHub через SSH

4. Документирование:

- Подготовлена полная документация по процессам
- Сохранены логи всех операций
- Оформлен подробный отчет

Все задачи выполнены в полном объеме, результаты работы зафиксированы в репозитории и готовы к проверке.